

安徽工业大学

毕业设计（论文）说明书

题 目

智能语音聊天机器人设计与实现

专 业

计算机科学与技术

班 级

计 225

姓 名

廖园

学 号

229074182

指导教师

王朋飞

二〇二六年五月二十七日

毕业设计（论文）独创性声明

本人郑重声明：所呈交的毕业设计（论文），是本人在指导教师指导下，进行研究工作所取得的成果。除文中已经注明引用的内容外，本毕业设计（论文）的研究成果不包含任何他人创作的、已公开发表或没有公开发表的作品内容。对本毕业设计（论文）所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。本毕业设计（论文）独创性声明的法律责任由本人承担。

毕业设计（论文）作者签名：

2026 年 5 月 27 日

本人声明：该毕业设计（论文）是本人指导学生完成的研究成果，已经审阅过论文的全部内容，并能够保证题目、关键词、摘要部分中英文内容的一致性和准确性。

毕业设计（论文）指导教师签名：

2026 年 5 月 27 日

摘 要

本课题面向中文语音交互场景，设计并实现了一套集文本对话、语音输入、实时语音通话、知识库增强与音色配置于一体的智能语音聊天机器人系统。针对传统聊天机器人在交互方式单一、语音链路割裂、上下文记忆不足以及实时性较弱等问题，论文以统一会话运行时为核心，构建了前后端协同的多模态智能交互平台。

系统后端采用 FastAPI 构建服务编排层，结合 PostgreSQL 与 pgvector 完成会话数据、记忆数据和知识库向量数据的存储；前端采用 React + Vite + TypeScript 构建交互界面，并通过 `SessionRuntime` 统一管理文本对话、分阶段语音链路和一体化实时语音链路。系统在语音交互部分同时支持 `integrated-realtime` 与 `split-chain` 两种执行策略，前者侧重低时延实时对话体验，后者侧重模块解耦、可替换性和工程可控性。为增强回答质量，系统引入检索增强生成机制，通过知识库切分、向量检索、关键词匹配与重排序生成知识上下文，再注入大语言模型完成回答生成。为提升交互自然度，系统还设计了语音打断、音频分片返回、音色克隆与多人格配置等能力。

论文围绕系统需求分析、总体架构设计、数据库设计、关键模块实现与系统测试展开。结果表明，该系统能够较好地完成多模态对话、知识增强问答与实时语音交互任务，具备较强的扩展性、可维护性和一定的工程应用价值。但系统在复杂噪声环境下的识别鲁棒性、长时对话一致性以及不同模型协同调度效率方面仍有进一步优化空间。

关键词：智能语音聊天机器人；多模态交互；检索增强生成；实时语音通话；统一会话运行时

Abstract

This study focuses on Chinese voice interaction scenarios and designs and implements an intelligent voice chatbot system integrating text dialogue, voice input, real-time voice calling, knowledge-base enhancement, and voice profile configuration. To address the limitations of traditional chatbots in interaction modality, speech pipeline continuity, contextual memory, and real-time responsiveness, the thesis takes a unified session runtime as the core and builds a multimodal interaction platform with coordinated frontend and backend components.

The backend is built with FastAPI and uses PostgreSQL together with pgvector to store session data, memory data, and vectorized knowledge-base data. The frontend is developed with React, Vite, and TypeScript, and uses SessionRuntime to manage text dialogue, a staged speech pipeline, and an integrated real-time speech pipeline in a unified manner. The system supports both integrated-realtime and split-chain execution strategies. The former emphasizes low-latency conversational experience, while the latter emphasizes modular decoupling, replaceability, and engineering controllability. To improve answer quality, the system introduces retrieval-augmented generation by combining document chunking, vector retrieval, keyword matching, and reranking before injecting the retrieved context into a large language model.

Experimental analysis and functional verification show that the system can effectively support multimodal dialogue, knowledge-enhanced question answering, and real-time speech interaction. The implemented architecture demonstrates good extensibility, maintainability, and practical engineering value. Nevertheless, there is still room for further improvement in speech recognition robustness under noisy conditions, long-session consistency, and collaborative scheduling efficiency across different models.

Keywords: intelligent voice chatbot; multimodal interaction; retrieval-augmented generation; real-time voice conversation; unified session runtime

目 录

1 绪论.....	1
1.1 研究背景.....	1
1.2 研究目的与意义.....	1
1.3 国内外研究现状.....	2
1.4 本文主要工作.....	2
1.5 论文结构安排.....	3
2 开发环境及相关技术介绍.....	4
2.1 开发环境.....	4
2.2 FastAPI 后端技术.....	4
2.3 React 前端技术.....	5
2.4 WebSocket 实时通信技术.....	5
2.5 检索增强生成技术.....	5
2.6 语音识别与语音合成技术.....	5
2.7 容器化与工程化技术.....	6
3 系统需求分析与数据库设计.....	8
3.1 系统功能需求分析.....	8
3.1.1 多模态交互需求.....	8
3.1.2 会话管理需求.....	8
3.1.3 知识增强需求.....	8
3.1.4 个性化配置需求.....	8
3.1.5 模型配置与路由需求.....	8
3.2 非功能需求分析.....	9
3.2.1 实时性.....	9
3.2.2 可扩展性.....	9
3.2.3 可维护性.....	9
3.2.4 可靠性.....	9
3.2.5 安全性与配置隔离.....	9
3.3 系统业务流程分析.....	10
3.4 数据库设计.....	10
3.4.1 会话与消息表.....	11
3.4.2 上下文记忆表.....	11
3.4.3 Provider 与模型目录表.....	11
3.4.4 个性化配置表.....	11
3.4.5 知识库相关表.....	12
3.4.6 审计与运行配置表.....	12
3.5 数据库设计小结.....	12

安徽工业大学毕业设计（论文）说明书

3.6 典型用例分析	12
3.6.1 文本问答用例	12
3.6.2 语音输入用例	12
3.6.3 实时语音通话用例	13
3.6.4 知识增强问答用例	13
4 系统总体设计	15
4.1 总体架构设计思路	15
4.2 统一会话运行时设计	16
4.3 双语音链路总体设计	16
4.4 事件驱动总体机制	17
4.5 知识增强总体设计	17
4.6 总体设计小结	18
4.7 系统部署与工程化设计	18
5 系统详细设计	20
5.1 统一会话运行时模块设计	20
5.2 分阶段语音链路设计	20
5.3 一体化实时语音链路设计	22
5.4 前端聊天中心页面设计	23
5.5 后端聊天服务模块设计	25
5.6 Provider 路由与模型回退设计	25
5.7 知识库与 RAG 模块设计	26
5.8 记忆管理模块设计	27
5.9 音色管理与克隆模块设计	28
5.10 实时 WebSocket 服务设计	29
5.11 测试用例对应的设计验证	31
5.12 关键设计特点总结	31
6 系统测试	32
6.1 测试目标	32
6.2 测试环境	32
6.3 测试内容	32
6.3.1 文本对话功能测试	32
6.3.2 语音输入功能测试	32
6.3.3 实时通话功能测试	33
6.3.4 知识增强功能测试	33
6.3.5 音色管理功能测试	33
6.4 测试用例设计	33
6.4.1 功能测试用例	33
6.4.2 对比与流程测试	34

安徽工业大学毕业设计（论文）说明书

6.5 测试结果分析.....	34
6.6 测试不足与改进方向.....	35
6.7 本章小结.....	35
结论.....	38

1 绪论

1.1 研究背景

近几年，大语言模型、语音识别和语音合成技术发展很快，人机交互的重心也在发生变化。许多系统已经不再满足于“输入文字、返回文字”的单一模式，而是开始追求更自然的交互过程，例如直接说话、连续追问、在系统播报过程中打断并切换话题等。对用户来说，这类能力会直接影响使用体验；对开发者来说，则意味着系统需要同时处理音频采集、语音理解、文本生成、结果播报和状态同步等多项任务。

从现有应用来看，文本问答系统已经较为成熟，但语音聊天机器人在工程实现上仍有不少难点。第一，文本输入、语音输入和实时通话常常分别由不同模块处理，缺少统一的会话抽象，前后端状态容易分散。第二，很多语音链路仍然采用“识别完成后再生成回答、生成完成后再播报”的串行方式，交互节奏偏慢。第三，当系统面向课程资料、项目文档或私有知识场景时，如果仅依赖通用大模型，回答往往容易停留在宽泛表述，难以稳定结合具体资料。

基于上述背景，设计一套兼顾文本输入、语音输入与实时语音通话能力，并能够结合知识增强、个性化音色与多轮记忆的智能语音聊天机器人系统，既有明确的应用价值，也适合作为本科阶段对系统设计、全栈开发与多模型协同能力的综合训练。

1.2 研究目的与意义

本课题的直接目标，是完成一套面向中文场景的智能语音聊天机器人系统，使其能够在统一框架下支持文本问答、语音输入和实时语音通话三种主要交互方式，并在同一套会话机制中管理消息、状态、指标与配置。

从系统设计角度看，本课题希望解决两个核心问题：一是如何用统一的运行时抽象把多种交互方式纳入同一套会话流程，二是如何在实时性与可控性之间取得平衡，使系统既能提供自然的语音交互体验，又保留阶段化链路便于调试、替换和分析的优势。

从教学与实践意义看，本系统并不是停留在单点功能验证，而是覆盖了前端交互、后端编排、数据库设计、知识检索、模型路由、语音能力接入和测试分析等多个环节。

安徽工业大学毕业设计（论文）说明书

将这些部分放在一个完整项目中统一实现，更能体现本科毕业设计对工程完整性和综合分析能力的要求。

1.3 国内外研究现状

在语音识别方面，技术路线已经从传统声学模型与语言模型分离的方式逐步转向端到端模型。Whisper 一类模型依靠大规模弱监督数据提升了跨语言泛化能力，FunASR、SenseVoice 等工程方案则更强调低延迟处理和中文场景下的部署灵活性。对于语音聊天机器人而言，识别模块除了准确率之外，还要兼顾流式能力、噪声环境表现以及与后续对话链路的衔接效率。

在对话大模型方面，相关研究已经证明大语言模型在问答、摘要、推理和多轮对话任务上具有较强能力。但把这类模型直接用于业务系统仍然会遇到一些现实问题，例如知识边界不稳定、输出内容难以约束、长上下文成本较高等。因此，工程系统通常不会只停留在“接入一个模型”，而是需要结合提示词、上下文组织和外部知识进行二次设计。

在检索增强方面，RAG 已成为提升回答可信度和场景适配性的常见方案。其基本思路是先从知识库中检索与问题相关的片段，再将结果与用户问题共同组织到提示上下文中，从而减少模型在专业资料问答中的脱离题意现象。对于需要结合项目文档、课程资料和私有知识的智能助手来说，这种方法具有较强实用性。

在记忆机制方面，越来越多的研究开始关注多轮对话中的上下文延续问题。若每一轮都简单拼接全部历史消息，系统既会面临上下文膨胀，也会增加无关信息干扰。因此，短期记忆、中期摘要和长期记忆的分层组织方式逐渐成为改进连续对话体验的重要方向。

在语音合成方面，当前 TTS 技术已经不再局限于基础播报，而是逐步向更自然的韵律控制、情感表达和音色定制发展。尤其是在音色克隆和零样本语音合成能力出现之后，语音机器人在“说得出来”之外，也开始追求“说得更像特定角色”。这使个性化语音交互成为可实现的工程能力，而不只是概念展示。

综合来看，现有研究和产品已经为智能语音聊天机器人提供了较好的技术基础，但在系统实现层面仍存在两类不足：一类是过度依赖单一闭源实时链路，导致分析与替换空间有限；另一类是模块虽然齐全，却缺少统一会话抽象，难以兼顾多种交互方式的一致性。本文的工作正是在这一背景下展开，重点尝试用统一会话运行时组织文本、语音与实时通话，并在同一系统中兼顾知识增强、记忆管理和个性化能力。

1.4 本文主要工作

安徽工业大学毕业设计（论文）说明书

围绕“智能语音聊天机器人设计与实现”这一课题，本文主要完成了以下几个方面的工作。

（1）在前端侧设计统一会话运行时，将文本对话、语音输入和实时语音通话纳入同一套状态机与事件流中，统一管理消息状态、转写文本、性能指标和异常处理。

（2）实现分阶段语音链路和一体化实时语音链路两种执行策略，使系统能够在低延迟体验与阶段可控性之间进行取舍，并为后续测试分析提供对比基础。

（3）完成知识库切片、向量检索、关键词匹配、重排序与上下文注入等模块设计，并将其与聊天服务、记忆管理和模型路由机制结合起来，形成较完整的对话处理闭环。

（4）实现会话管理、消息持久化、上下文记忆、人格配置、音色管理与音色克隆等配套功能，并结合测试代码与系统运行结果，对关键流程的可用性和工程完整性进行了验证。

这些工作共同构成了本文的主要实现内容，也构成后续系统设计、详细设计和测试分析章节的展开基础。

1.5 论文结构安排

全文共分为七章。第 1 章介绍课题背景、研究目的、相关研究现状以及本文主要工作。第 2 章说明系统开发环境与关键技术。第 3 章进行需求分析和数据库设计。第 4 章给出系统总体设计。第 5 章围绕关键模块展开详细设计与实现说明。第 6 章介绍系统测试过程、测试结果与后续改进方向。第 7 章对全文工作进行总结，并给出后续完善思路。

2 开发环境及相关技术介绍

2.1 开发环境

本系统采用前后端分离模式进行开发。前端主要负责页面展示、用户交互、语音采集与状态反馈，后端主要负责会话管理、模型调用、知识检索、消息持久化及实时 WebSocket 通信。系统主要开发环境如下：

前端开发框架：React、Vite、TypeScript

后端开发框架：FastAPI

数据访问层：SQLAlchemy

数据库：PostgreSQL

向量扩展：pgvector

实时通信方式：WebSocket

语音识别方式：浏览器 ASR 与后端 ASR 双路径

文本生成与实时模型：多 Provider 路由调用

语音合成：TTS 接口调用与音色克隆能力

部署方式：Docker 容器化部署

从工程结构上看，系统后端按功能模块划分为聊天、上下文记忆、知识库检索、实时语音通信、音色管理、人格配置、模型目录和系统设置等模块；前端则包括页面层、状态管理层、语音运行时层以及接口访问层。

前端实际路由包括聊天中心、上下文管理、模型厂商与模型管理、链路配置、人格配置、音色管理、知识库管理和系统配置八个页面，分别对应 ChatCenterPage、ContextPage、ProviderModelPage、ChainPage、PersonaPage、VoicePage、KnowledgePage 和 SettingsPage。后端实际路由则围绕 chat、context_memory、knowledge_rag、provider_registry、model_catalog、chain_config、persona、voice、realtime_ws、call_config 和 system_settings 等模块组织，论文后续章节均按照这一实现结构展开。

2.2 FastAPI 后端技术

FastAPI 是基于 Python 的现代异步 Web 框架，具有开发效率高、异步支持完善、与 Pydantic 结合紧密等优点。对于本课题而言，FastAPI 主要承担三个方面的任务：一

是提供会话、消息、知识库、音色和配置管理等 REST API；二是提供实时语音对话所需的 WebSocket 接口；三是作为各类模型调用与数据库访问的统一编排层。

系统在 `main.py` 中统一注册了聊天、知识库、上下文记忆、实时通信、音色、人格、模型目录等多个路由模块，并在启动阶段自动写入默认 Provider 配置和默认通话配置。这种模块化设计便于后续迭代，也符合软件工程中高内聚、低耦合的设计原则。

2.3 React 前端技术

前端采用 React + Vite + TypeScript 构建。React 便于组织复杂交互界面，TypeScript 能提升工程可维护性和类型安全，Vite 则使开发与构建速度更快。系统前端不仅负责传统聊天界面，还承担语音采集、实时事件监听、消息状态更新、链路切换、指标展示等复杂交互任务，因此需要一套统一的运行时抽象对这些行为进行管理。

2.4 WebSocket 实时通信技术

由于实时语音交互需要客户端与服务端之间维持持续双向通信，因此系统采用 WebSocket 作为语音链路的核心通信方式。相较于传统的 HTTP 请求响应模式，WebSocket 更适合传输音频片段、局部识别结果、增量文本和音频分片等连续事件。

在一体化实时语音链路中，前端通过 WebSocket 向服务端持续发送音频片段，服务端则将识别结果、文本回复、音频开始与停止事件、打断事件和指标事件持续回传给前端。这种机制使系统能够实现接近实时的人机对话体验。

2.5 检索增强生成技术

检索增强生成技术是本系统提升回答准确性的重要手段。系统在用户提出问题后，先从知识库中执行切片检索，再将召回结果整理为知识上下文，与对话历史和人格提示词一起注入模型上下文中生成回答。这样做的优势在于，模型不再完全依赖参数记忆，而是可以结合项目文档、课程资料或业务知识库完成更具依据的回答。

本系统的检索机制不是单纯的向量搜索，而是结合向量检索、关键词匹配和重排序思路进行综合处理。该设计有助于减少错误召回带来的跨文档串扰，提高问答结果的稳定性和可解释性。

2.6 语音识别与语音合成技术

在语音识别方面，系统支持浏览器语音识别与后端识别两种路径。浏览器识别适合快速启动与低成本场景，后端识别适合更稳定的服务端处理与统一管理。通过“双路径识别”设计，系统在不同浏览器能力和不同部署环境下都能保持较好的兼容性。

在语音合成方面，系统不仅支持基础的语音播报，还支持音色样本上传、自动转写参考文本、克隆音色并生成自定义语音配置。这使系统不仅能“说出来”，还能在一定程度上“按指定角色说出来”，增强了系统的交互自然性和个性化程度。

2.7 容器化与工程化技术

为了便于系统部署与环境复现，本项目同时引入了 Docker 与数据库迁移机制。前后端均提供独立 Dockerfile，后端通过 Alembic 管理数据库结构演进，保证在不同开发阶段能够持续更新表结构而不破坏既有数据。对于本科毕业设计而言，这一部分虽然不是算法创新点，但它能显著提高系统的可移植性、可部署性和工程完整性，也体现了现代软件项目开发中的规范化思路。

3 系统需求分析与数据库设计

3.1 系统功能需求分析

结合课题目标与项目实现情况，系统功能需求主要包括以下几个方面。

3.1.1 多模态交互需求

除文本、语音和实时通话外，系统还需要支持图片生成类多模态输出。当用户输入“生成图片”“画一张”等明确图像生成意图时，后端聊天服务会识别图像提示词，并通过配置好的图片生成 Provider 返回生成结果。该能力不改变主聊天链路，但扩展了系统从文字与语音交互到图文输出的边界。

系统需要同时支持文本输入、语音输入和实时语音通话三种模式。文本输入主要用于传统聊天和知识问答；语音输入主要用于将用户语音转写后发起一轮问答；实时语音通话则要求系统支持边说边听、回复播报以及用户插话打断。

3.1.2 会话管理需求

系统需要能够创建、恢复和结束会话，并对用户消息、助手消息以及中间状态进行统一管理。会话不仅要保存消息内容，还要保留链路配置、人格配置、音色配置和是否启用知识库等信息，以支持后续恢复和复现。

3.1.3 知识增强需求

系统需要支持知识库创建、文档上传、文档切片、向量存储与检索增强问答，使模型回答不再局限于通用知识，而能够结合本地资料完成更准确的回答。

3.1.4 个性化配置需求

系统需要支持人格模板、音色模板和链路配置的独立管理，并能够在不同会话中灵活组合。这既是系统完整性的体现，也能提升用户体验和扩展性。

3.1.5 模型配置与路由需求

由于不同任务适合不同模型，系统需要具备多 Provider 和多模型目录管理能力，支持主模型和备用模型配置，提升系统的容错能力与可扩展能力。

3.2 非功能需求分析

除功能需求外，系统还需满足以下非功能需求。

3.2.1 实时性

语音对话系统最核心的非功能需求之一是响应时效。系统应尽可能缩短从用户说话结束到模型开始返回结果的时间，并支持在播报阶段对用户插话进行快速响应。

3.2.2 可扩展性

系统需要支持新模型、新音色、新人格和新知识库的快速接入，因此在架构上应尽量避免将业务逻辑与具体厂商接口强绑定。

3.2.3 可维护性

系统模块较多，涉及前端状态管理、后端接口、数据库和模型调用。若缺少清晰边界，后期维护成本会很高。因此系统在设计中采用模块分层与事件驱动思想，提升维护性。

3.2.4 可靠性

系统在模型调用失败、语音识别失败或音色克隆失败时，需要给出明确错误提示，并尽量保留当前会话状态和历史数据，以避免交互过程完全中断。

3.2.5 安全性与配置隔离

由于系统需要管理多家模型服务提供方的 API 信息，因此配置数据不能直接散落在前端页面或普通文本中。项目中通过独立的 Provider 配置表保存服务地址、超时时间和密钥信息，并在后端统一处理模型访问逻辑。这样既降低了前端暴露敏感配置的风险，也使模型接入方式更加集中可控。

3.3 系统业务流程分析

系统主要业务流程包括文本对话流程、语音输入流程和实时通话流程。

文本对话流程为：用户输入文本，前端调用会话运行时发起文本轮次，请求后端聊天服务，后端结合历史上下文和知识检索结果生成回答，再将文本和可选音频返回前端显示与播放。

语音输入流程为：用户在前端触发录音，浏览器识别或后端识别得到文本，再将文本作为一轮输入提交至聊天服务，后续处理流程与文本对话一致。

实时通话流程为：前端持续采集音频并通过 WebSocket 传输给后端，后端根据事件时机执行局部转写、整句提交、回复生成和音频合成，同时向前端持续回传状态、文本和音频事件。若用户在播报阶段再次讲话，系统触发打断机制，终止当前回复并转入下一轮处理。

3.4 数据库设计

如图 3-1 所示，系统数据库并不是围绕单一聊天记录设计，而是同时覆盖了会话、消息、记忆、知识库、模型配置与运行审计等多类数据对象。这种结构说明本系统不仅需要完成问答功能，还需要为知识增强、链路切换和运行追踪提供数据支撑。

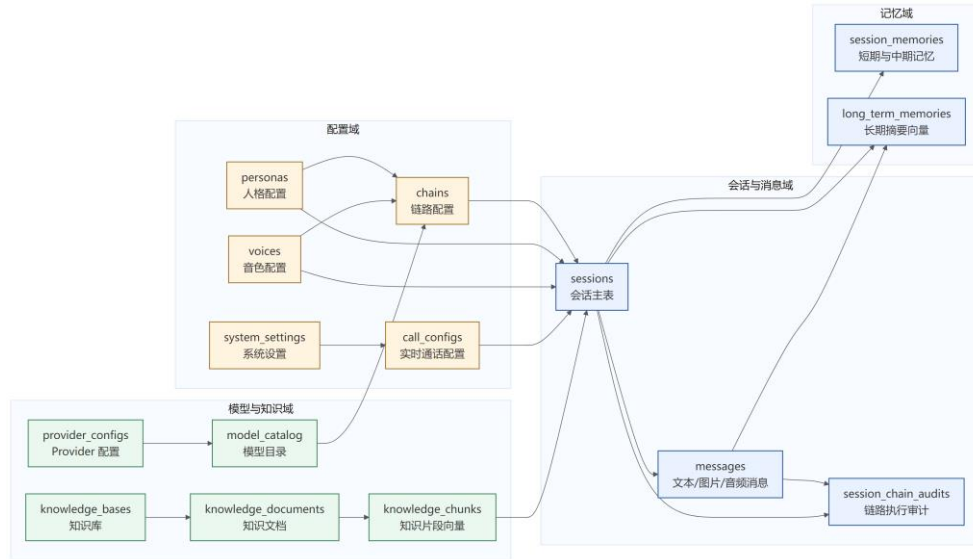


图 3-1 数据库 E-R 关系图

系统采用 PostgreSQL 作为主数据库，并结合 pgvector 支持向量数据存储。根据系统功能需要，数据库主要由以下几类核心表构成。

3.4.1 会话与消息表

‘sessions’表用于保存会话基本信息，包括会话标题、链路、人格、音色、知识库开关、状态以及创建更新时间等。

‘messages’表用于保存每条用户消息和助手消息，支持文本、图片和音频等多种内容类型，同时保留消息元数据和音频地址。

这两张表共同构成系统历史对话的基础数据层，支撑会话恢复、对话展示和轮次审计。

3.4.2 上下文记忆表

‘session_memories’表用于保存当前会话的短期记忆和中期摘要，主要服务于多轮对话连续性管理。

‘long_term_memories’表用于保存长期记忆摘要及其向量表示，以支持后续语义召回。

这种记忆结构并不是简单保存全部历史文本，而是通过短期、中期、长期分层管理方式降低上下文冗余问题。

3.4.3 Provider 与模型目录表

`provider\_configs` 表用于保存不同模型服务提供方的基础信息，包括服务地址、加密后的密钥和状态。

`model\_catalog` 表用于保存各 Provider 下的模型名称、类型、启用状态和附加元数据。

通过这两张表，系统能够实现多模型可配置和多厂商可切换。

3.4.4 个性化配置表

`personas` 表保存人格配置，`voices` 表保存音色配置，`chains` 表保存链路映射配置。

三者彼此独立，但可以在会话创建时进行组合绑定，从而实现“同一套系统，不同交互风格”的目标。

3.4.5 知识库相关表

`knowledge\_bases` 表用于保存知识库基本信息，`knowledge\_documents` 表保存上传文档信息，`knowledge\_chunks` 表保存切片内容、元数据与向量。

这是系统实现检索增强生成的关键数据基础。

3.4.6 审计与运行配置表

`call\_configs` 表用于保存默认实时通话配置和回退模型配置。

`session\_chain\_audits` 表记录链路执行模式、成功状态、失败原因和跟踪信息，为系统调试、实验分析和问题追踪提供依据。

3.5 数据库设计小结

从数据库结构来看，本系统并非单一聊天记录系统，而是围绕会话、记忆、知识库、模型配置与运行审计构建了较完整的数据支撑体系。这一设计使系统不仅能够完成问答任务，还具备一定的实验平台和工程管理属性。

3.6 典型用例分析

为了更清晰地说明系统需求，本文进一步选取几个典型使用场景进行分析。

3.6.1 文本问答用例

用户在聊天中心输入文本问题，系统创建或切换到指定会话后，将文本提交给后端。后端根据会话对应的人格、链路、知识库开关和模型配置组织上下文，并返回文本回答及可选音频。该用例体现了系统的基础问答能力，也是其它语音场景最终会收敛到的核心处理路径。

3.6.2 语音输入用例

用户点击语音输入后开始说话。系统根据当前设备能力优先采用浏览器识别，若浏览器不支持则回退到后端录音识别。识别得到文本后，系统自动将文本作为一轮消息提交，并返回对应回答。该用例体现了系统在兼容性和易用性方面的设计目标。

3.6.3 实时语音通话用例

用户启动实时通话模式后持续讲话，系统实时接收音频并返回部分转写、最终转写、助手文本与语音播报结果。如果在助手播报期间用户再次开口，系统将执行打断逻辑，结束当前回复并开始新的轮次。该用例能够集中体现系统在实时性、状态切换和事件驱动设计方面的特点。

3.6.4 知识增强问答用例

为了更直观地说明本系统面向的使用环境，本文在需求与典型用例分析之后补充多模态交互应用场景示意图，如图 3-2 所示。该图用于说明文本、语音输入和实时通话在同一学习或咨询场景中的协同关系，不替代系统真实界面与真实链路图。



图 3-2 多模态交互应用场景示意图（AI 示意图）

用户在导入知识库文档后，提出与文档内容相关的问题。系统先执行知识检索，召回文档切片并构造知识上下文，再交由文本模型生成回答。该用例主要体现系统相较于普通聊天机器人的差异化能力。

4 系统总体设计

如图 4-1 所示，系统整体采用自上而下的分层结构。前端表现层负责交互呈现，会话运行层负责统一状态与事件，策略执行层负责组织不同语音链路，服务编排层负责模型调用与业务处理，底层数据与配置层则提供持久化和检索支撑。

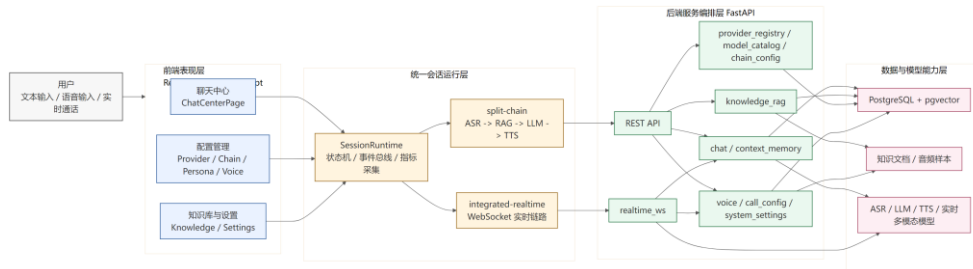


图 4-1 系统总体架构图

4.1 总体架构设计思路

为了更具体地说明各主要功能在系统中的组织关系，本文进一步给出系统功能模块划分，如图 4-2 所示。该图强调的是面向使用场景的功能构成，而不是底层实现层次。

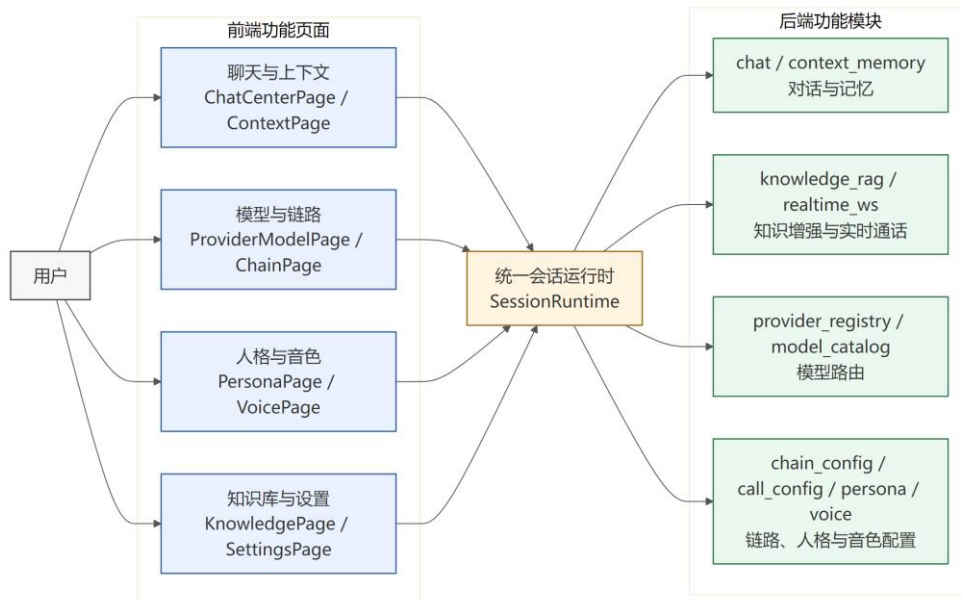


图 4-2 系统功能模块图

从图 4-2 可以看出，聊天中心、语音输入、实时通话、知识库管理以及人格与音色管理等前端功能，最终都通过统一会话运行时与后端服务进行协同。这种组织方式使系统既能保持功能边界清晰，又能避免不同页面各自维护一套独立的交互逻辑。

本系统遵循分层设计、统一抽象和策略可替换的设计思想。系统总体上可以划分为表现层、会话运行层、策略执行层、服务编排层和数据配置层五个层次。

表现层由前端页面构成，负责展示对话界面、配置界面、知识库管理界面和系统设置界面，并采集用户输入。

会话运行层以前端 `SessionRuntime` 为核心，负责统一会话生命周期管理、事件收发、状态切换和指标采集。

策略执行层负责组织不同交互策略，包括一体化实时链路和分阶段链路。

服务编排层由 FastAPI 提供，负责会话接口、模型调用、知识检索、实时 WebSocket 通道和音色处理等。

数据与配置层由 PostgreSQL、pgvector 以及文件存储共同构成，负责持久化会话数据、知识向量、音色样本和配置数据。

4.2 统一会话运行时设计

统一会话运行时是本系统的重要设计亮点。前端通过 `SessionRuntime` 将文本对话、语音输入和实时语音通话统一在同一套状态管理与事件流体系下。该运行时内部维护当前会话标识、当前模式、工作状态、消息列表、转写文本以及上一轮指标等信息，并对外暴露 `startListening()`、`stopListening()`、`sendTextTurn()`、`interruptCurrentReply()` 和 `applyLiveConfig()` 等统一接口。

这种设计带来的直接优势有三点。第一，界面层无需关心底层是浏览器 ASR、后端 ASR 还是实时 WebSocket，只需消费统一事件。第二，不同语音链路能够在同一界面下平滑切换，减少重复实现。第三，系统能够统一记录首字延迟、首音延迟、检索延迟、打断延迟等指标，为后续测试分析提供基础。

4.3 双语音链路总体设计

如图 4-3 所示，系统并没有把语音交互理解为单一路径，而是在统一会话框架下同时容纳分阶段链路和实时链路。用户输入、识别结果、文本生成、语音播报以及打断控制都围绕同一套事件流展开，这也是系统能够平滑切换不同交互方式的基础。



图 4-3 双语音链路处理流程图

系统支持两种语音链路：

`integrated-realtime` 链路

该链路通过持续 WebSocket 连接完成音频上传、局部识别、文本回复、音频返回与打断控制，适合实时对话与近似电话式交互场景。

`split-chain` 链路

该链路采用“语音识别 -> 文本理解 -> 知识增强 -> 文本生成 -> 语音合成”的分阶段方式实现，适合调试、分析和阶段性优化。

二者并不是完全独立的两套系统，而是在同一会话运行时框架下由不同策略模块实现。该设计兼顾了实时体验和工程可控性，是本文系统架构的核心特征之一。

4.4 事件驱动总体机制

为了统一不同链路的输出形式，系统采用事件驱动机制作为前后端协作核心。系统中的关键事件包括：

```
`session.ready`  
`session.updated`  
`session.state.changed`  
`user.transcript.partial`  
`user.transcript.final`  
`assistant.text.delta`  
`assistant.text.final`  
`assistant.audio.started`  
`assistant.audio.delta`  
`assistant.audio.stopped`  
`assistant.interrupted`  
`metrics.turn`  
`metrics.retrieval`
```

通过这些统一事件，前端能够实时更新消息内容、界面状态和指标信息，而后端也能够不同链路中维持一致的协议约定。这种机制提高了系统的清晰度和扩展性。

4.5 知识增强总体设计

在总体设计说明之外，本文增加检索增强生成工作流程示意图，如图 4-4 所示，用于帮助理解检索门控、向量召回、重排序与上下文注入之间的关系。

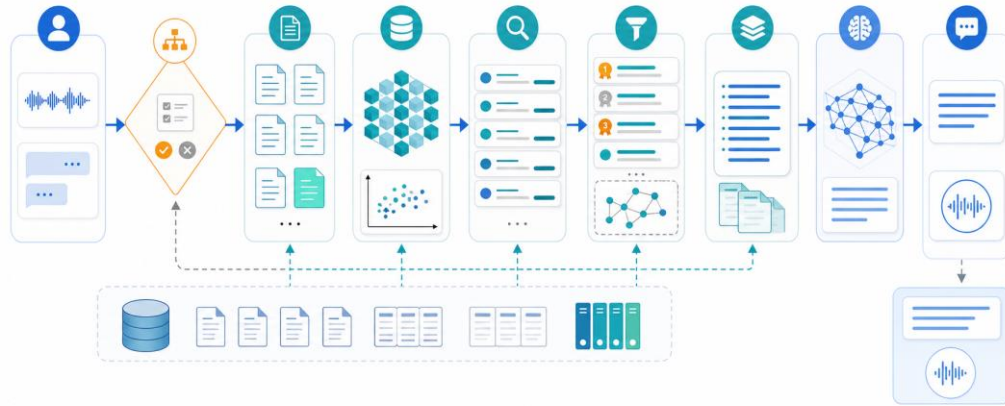


图 4-4 检索增强生成工作流程示意图（AI 示意图）

系统在总体设计中将知识增强能力视为对话系统的一等能力，而不是额外附加功能。当用户提问时，后端会先根据问题内容检索知识库，获得候选切片，再将其整理为知识上下文，与对话历史和人格提示词共同提供给文本生成模块。这样既提高了系统的回答准确性，也使知识来源更可追踪。

4.6 总体设计小结

总体而言，本系统不是简单拼接若干模型接口，而是从统一会话、统一事件协议和可替换链路三个层面完成架构设计。这样的系统既可以作为面向实际交互的应用原型，也可以作为面向实验分析和教学演示的工程平台。

4.7 系统部署与工程化设计

从工程落地角度看，本系统采用前后端分离与容器化相结合的部署思路。前端通过 Vite 完成构建后提供静态资源，后端以 FastAPI 服务形式对外提供 REST API 与 WebSocket 接口，数据库层使用 PostgreSQL 保存会话、消息、配置、知识库和向量数据。该部署方式能够将界面、服务和数据三类职责分离，便于后续单独升级或排查问题。

系统的关键配置统一由后端管理，前端只通过接口读取必要状态，不直接保存模型密钥、数据库连接串等敏感信息。对于模型服务、音色服务和知识检索服务，系统在后端设置 Provider 配置与模型目录，使调用入口、模型名称、超时时间和可用状态能够集中维护。这样既减少了前端暴露敏感参数的风险，也便于在模型不可用时进行回退或替换。

安徽工业大学毕业设计（论文）说明书

在日志与审计方面，系统通过会话链路审计记录每轮请求的执行模式、链路名称、是否成功、失败原因和关键 **trace** 信息。该设计可以帮助开发者定位模型调用失败、知识检索异常、语音合成失败等问题，也能为第 6 章系统测试中的结果分析提供依据。

为了让论文表达更直观，后续定稿时可在本节配套加入系统总体架构图、双链路语音交互图、数据库 E-R 图和实时通话时序图。本文初稿先以文字和表格说明为主，后续可根据答辩材料需要补充图示。

5 系统详细设计

5.1 统一会话运行时模块设计

在前端实现中，SessionRuntime 负责承接界面层与底层执行策略之间的衔接工作。该模块并不直接关心具体由哪一种语音链路执行，而是先把会话、消息、状态、转写文本和性能指标组织成统一的运行时对象，再向上层页面暴露一致的事件与快照。这样做的目的，是让页面逻辑不必分别适配文本问答、语音输入和实时通话三种流程。

从处理流程看，运行时会根据底层事件持续修正前端状态。例如局部转写到达时，会更新临时文本；助手开始输出时，会将对应消息标记为生成中；回复被中断时，又会把该轮消息改为 interrupted 状态。正是因为有了这层统一抽象，前端页面才能以较小的改动接入不同策略，并较真实地呈现一轮对话从“监听”到“思考”、再到“播报”的完整变化过程。

对应实现中，SessionRuntime 通过统一事件入口同步转写文本、助手增量回复与轮次指标，关键片段如下：

```
if (type === "user.transcript.partial") {
  this.snapshot.transcript = data.text;
  this.addUserMessage(data.text, true);
}
if (type === "assistant.text.delta") {
  this.addOrUpdateAssistantMessage(data.text, false);
}
```

5.2 分阶段语音链路设计

分阶段语音链路由 SplitChainStrategy 实现，其设计目的在于把识别、检索、生成和播报拆分为相对独立的阶段，以便分别替换模型、观察中间结果并分析各阶段耗时。对于需要做实验比较或需要更强可控性的场景，这种设计比完全黑盒化的一体化实时接口更适合作为工程方案。

在具体流程上，系统会先根据环境能力判断使用浏览器原生识别还是后端识别，再将识别文本提交给聊天服务，之后依次进入知识增强、文本生成和语音播报环节。为了避免助手播报内容被重新当作用户输入，链路中加入了回声抑制判断；当系统处于 thinking 或 speaking 状态且检测到新的有效发言时，还会触发插话打断并启动下一轮

安徽工业大学毕业设计（论文）说明书

处理。这样一来，分阶段链路虽然不追求极限实时性，但在可分析性和稳定性上具有明显优势。

链路配置界面用于把一体化链路与可拆分链路绑定到具体模型、人格和音色，相关页面如图 5-4 所示。

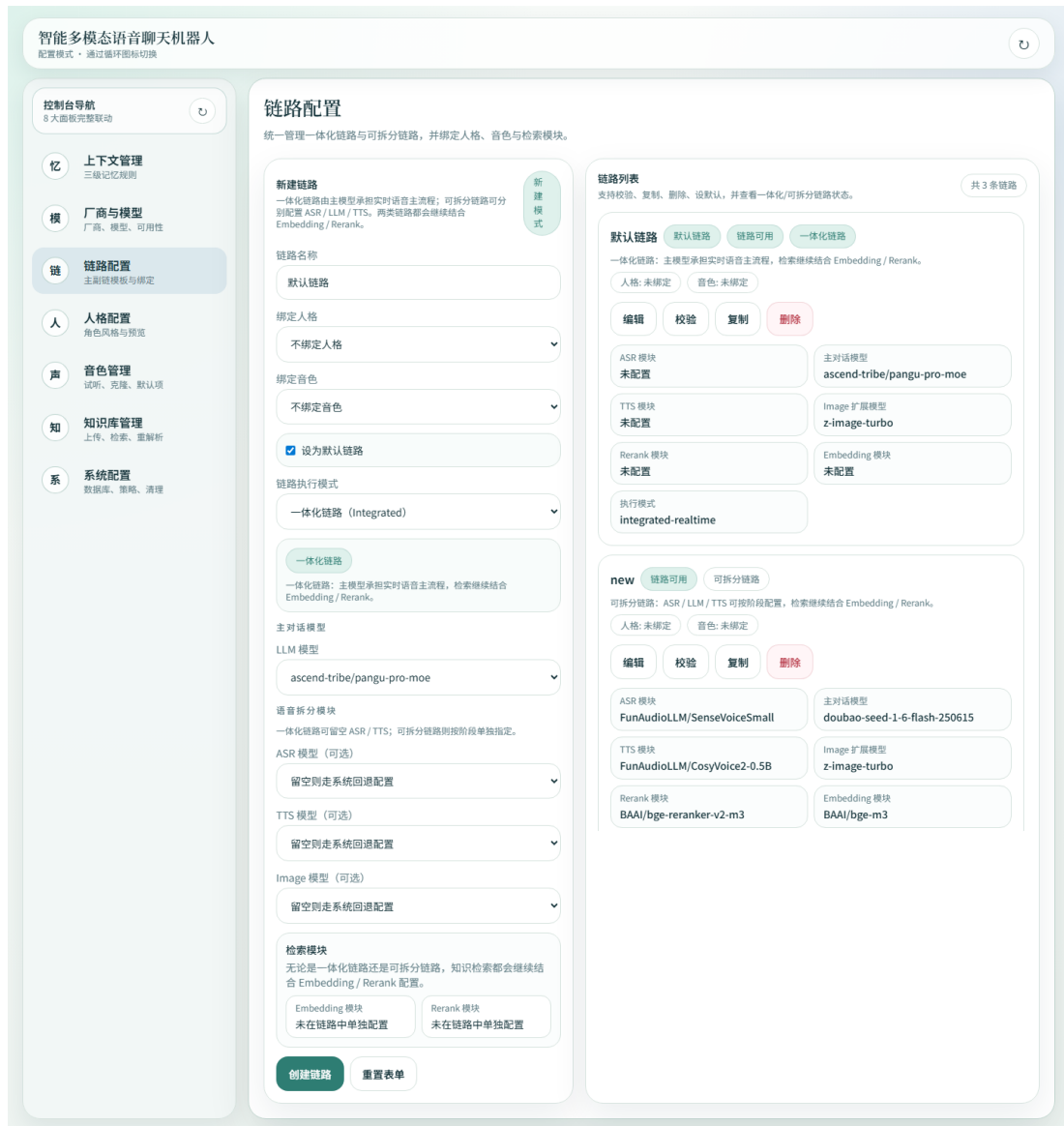


图 5-4 链路配置页面

SplitChainStrategy 在浏览器 ASR 与后端 ASR 之间自动切换，并在识别阶段提前执行插话判断：

```
if (cfg.splitAsr === "backend") {  
  await this.startBackendAsr(ctx);  
  return;  
}  
try {  
  await this.startBrowserAsr(ctx);  
} catch {  
  await this.startBackendAsr(ctx);  
}
```

5.3 一体化实时语音链路设计

图 5-1 展示了一体化实时语音链路中的主要时序关系。可以看出，该链路并不是在用户说完之后才统一处理，而是通过持续传输音频块、局部转写、整句判定和回复生成的并行协同，尽量压缩等待时间并提升插话响应能力。

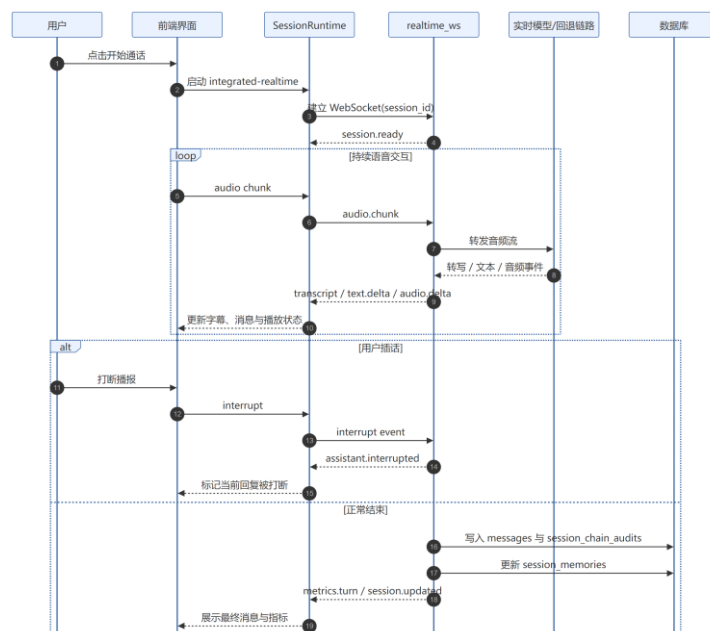


图 5-1 实时语音通话时序图

一体化实时语音链路由 `IntegratedRealtimeStrategy` 实现，其核心目标是尽可能缩短“用户说话”与“系统给出反馈”之间的等待时间。客户端通过 `WebSocket` 与后端保持持续连接，并把采集到的音频块按统一格式传输到服务端。服务端在接收音频的同时，分阶段完成局部转写、整句判定、回复生成与音频合成，从而减少传统串行链路中的等待环节。

为了让实时通话更接近日常交流，该链路引入了基于音频能量的讲话检测机制。系统会结合均方根能量、连续帧数和当前播报状态判断是否出现新的用户发言；如果检测

到用户在助手播报阶段再次开口，前端会主动向服务端发送取消事件，中止当前回复并切入新的轮次处理。

与分阶段链路相比，一体化实时链路的优势在于交互节奏更紧凑、插话更自然，更适合强调陪伴感和连续对话体验的场景；但它对网络稳定性、音频处理和后端协同提出了更高要求，调试成本也相对更大。这也是本文保留两种链路并行设计的重要原因。

`IntegratedRealtimeStrategy` 使用 `WebSocket` 持续上送音频块，并把服务端事件还原为统一运行时事件：

```
const ws = new WebSocket(`${getWsBase()}/ws/realtime-chat?session_id=${sessionId}`);
ws.onmessage = (event) => {
  const data = JSON.parse(String(event.data));
  if (data.event === "assistant.text.delta") {
    ctx.emit("assistant.text.delta", { text: String(data.text ?? "") });
  }
};
```

5.4 前端聊天中心页面设计

从页面结构看，前端并非只有聊天中心一个页面。`ChatCenterPage` 负责主对话与实时语音交互，`ContextPage` 负责上下文记忆配置，`ProviderModelPage` 负责模型厂商与模型目录管理，`ChainPage` 负责链路组合，`PersonaPage` 与 `VoicePage` 分别负责人格和音色配置，`KnowledgePage` 负责知识库文档管理，`SettingsPage` 则承担全局配置维护。这种页面划分与系统实际路由保持一致，也使论文中的模块说明能够对应到真实代码结构。

聊天中心页面是系统的核心交互入口。根据项目实现，页面主要包含会话列表、消息展示区域、输入区域、模式切换区域以及运行状态与指标展示区域。页面会根据运行时返回的状态变化，将“待机”“监听中”“思考中”“播报中”和“异常”等状态实时呈现给用户。

页面还维护了当前会话、当前链路模式、是否正在监听、当前转写文本以及最近一轮的性能指标。例如当系统收到 ``metrics.turn`` 事件时，页面会展示检索耗时、首字延迟、首音延迟、LLM 耗时、TTS 耗时和总耗时等信息。这说明本系统并不只是完成“能聊”的功能，还额外关注“聊得怎么样”的可观测性设计。

聊天中心页面承担主交互入口，其真实界面如图 5-2 所示。

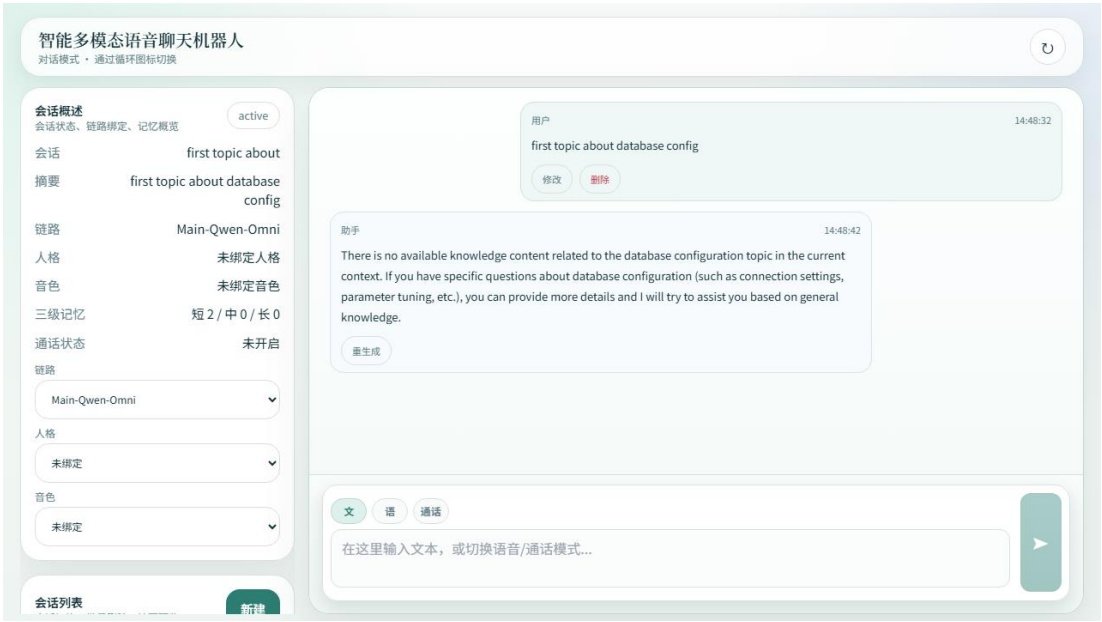


图 5-2 聊天中心页面

人格配置页用于维护角色定位、语气、严格程度和默认人格规则，其界面如图 5-3 所示。



图 5-3 人格配置页面

ChatCenterPage 直接订阅 SessionRuntime 事件，并把轮次性能指标同步到页面状态：

```
if (event.type === "metrics.turn") {
  setRuntimeMetrics({
    turnId: event.payload.turn_id,
    firstTextLatencyMs: event.payload.first_text_latency_ms,
    totalLatencyMs: event.payload.total_latency_ms,
  });
}
```

5.5 后端聊天服务模块设计

后端入口在 `main.py` 中集中注册各业务路由，其中 `chat` 模块负责会话与回复生成，`context_memory` 模块负责记忆维护，`knowledge_rag` 模块负责文档解析、切片、向量化和检索测试，`provider_registry` 与 `model_catalog` 负责模型服务配置，`chain_config`、`persona`、`voice` 分别负责链路、人格和音色，`realtime_ws` 负责实时语音 `WebSocket` 通道，`call_config` 与 `system_settings` 则负责默认通话配置和系统级参数。这些模块共同构成了论文所描述的服务编排层。

后端聊天服务承担了从用户输入到最终回复输出的主要编排工作。系统接收到一轮输入后，会先解析当前会话对应的人格、音色、链路和模型配置，再读取最近的历史消息组织上下文；如果当前问题需要知识增强，则进一步发起检索，并把检索到的资料片段同用户问题一起送入文本生成链路。

在回复生成完成后，系统还会统一记录链路名称、`Provider` 信息、回退原因、知识命中情况和音色配置等运行数据。也就是说，聊天服务并不只是“把答案生成出来”，而是把这一轮回答的来源、路径和关键配置同时保存下来，便于后续调试、测试与论文分析。

聊天服务模块在发起回复前先完成知识检索决策，再组织上下文、`RAG` 片段与会话配置生成统一请求：

```
decision = should_retrieve_knowledge(user_text, image_prompt=image_prompt)
if decision.decision == "retrieve":
    knowledge_text, rag_trace, rag_reason, retrieval_timed_out = await
    _retrieve_knowledge_text(
        db, rag_query, chain_mapping=getattr(runtime, "chain_mapping", None)
    )
```

5.6 Provider 路由与模型回退设计

为了避免模型调用与业务逻辑紧耦合，系统把 `Provider` 配置、模型目录和默认通话配置统一抽离出来，通过路由层决定不同环节所使用的服务接口。语音识别、文本生

成和语音合成并不要求来自同一厂商，这为后续切换模型版本、补充备用方案和比较不同服务效果提供了基础。

这种设计的实际意义在于，后端不需要因为某一类模型服务变化而重写整条业务链路。当部分接口响应异常或不可用时，系统能够更方便地回退到备用方案，从而提高整体可维护性。

厂商与模型管理页面把 Provider 配置、连通性测试与模型同步放在统一入口中，相关界面如图 5-5 所示。



图 5-5 厂商与模型管理页面

Provider 路由通过测试连通性和同步远端模型，把模型目录与业务调用解耦：

```
ok, msg = await test_connectivity(db, provider_name)
row.status = "connected" if ok else "error"
remote_models = await list_models_remote(db, provider_name)
for item in remote_models:
    model_name = str(item.get("id") or item.get("model") or "")
```

5.7 知识库与 RAG 模块设计

在多模态扩展方面，聊天服务还包含图片生成意图识别逻辑。当用户文本匹配 /image、/draw 或“生成图片”等表达时，系统会进入图片生成链路；如果用户明确要求结合知识库资料生成图片提示词，RAG 检索结果也会作为可选参考注入提示中。这样，知识增强不仅服务于文字问答，也可以辅助图像生成场景。

知识增强模块负责把“原始文档”转化为“可被模型调用的上下文”。文档导入后，系统会先完成切片、向量化和元数据存储；当用户提问时，再结合向量检索、关键词匹配和重排序结果选出较相关的片段，最后把这些内容与会话历史一起送入文本模型。

从作用上看，这一模块解决的是“模型会说，但未必知道本项目资料”的问题。将知识检索结果显式注入对话上下文后，系统在回答项目配置、课程文档和私有资料相关问

安徽工业大学毕业设计（论文）说明书

题时更容易给出贴近材料的说明，而不是停留在泛化表述。与此同时，检索命中条数、耗时和来源信息也能够被记录下来，为测试分析提供依据。

知识库管理页面用于维护知识库、上传文档、设置检索参数并执行检索测试，界面如图 5-6 所示。

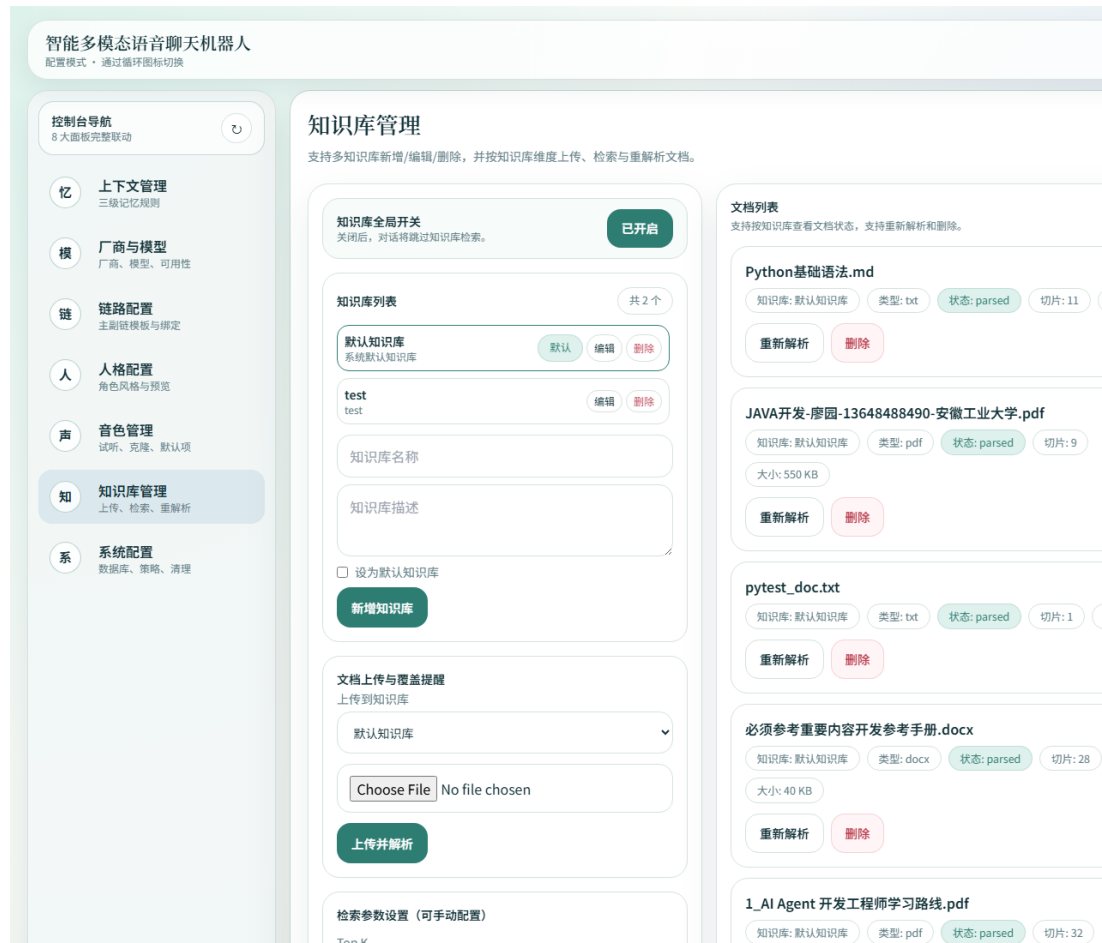


图 5-6 知识库管理页面

文档进入知识库后会先完成切片、向量化和分块持久化，再被用于后续检索：

```
chunks = split_chunks(full_text, chunk_size=chunk_size, overlap=overlap)
vectors = await embedding(db, embedding_provider, embedding_model, chunks)
for idx, chunk in enumerate(chunks):
    db.add(KnowledgeChunkEntity(document_id=document.id, chunk_index=idx,
    content=chunk, embedding=vectors[idx][:8]))
```

5.8 记忆管理模块设计

记忆管理模块主要面向多轮对话中的连续性问题。系统没有简单地把全部历史消息原样拼接进提示词，而是区分短期记忆、中期摘要和长期记忆三类信息：短期记忆保留最近几轮关键上下文，中期摘要压缩阶段性内容，长期记忆则以可检索形式保存用户偏好和重要事实。这种结构既控制了上下文长度，也为长时对话中的信息回忆提供了支撑。

安徽工业大学毕业设计（论文）说明书

上下文管理页面把三级记忆参数、当前会话快照、导出与压缩操作集中到一个页面中，界面如图 5-7 所示。



图 5-7 上下文管理页面

后台通过短期原文、中期摘要和长期记忆的分层重算，控制上下文长度并保留关键事实：

```
short = payload[-memory.short_turns:]
mid_raw = older[-memory.mid_turns:]
memory.mid_summary = [{"summary": summarize_text(x["text"]), "message_id":
x["message_id"]} for x in mid_raw]
for item in long_raw[-memory.long_turns:]:
    vec = await _embed_summary(db, summarize_text(item["text"]))
```

5.9 音色管理与克隆模块设计

系统音色模块支持音色列表管理、音色创建、音色重命名、音色预览、样本上传和音色克隆等功能。用户上传参考语音后，系统首先检查样本文件格式、采样率、位宽和时长是否符合要求；若符合要求，则可进一步调用语音服务提供方完成参考音色注册，并将克隆后的音色 URI 保存到数据库配置中。

这一模块的工程价值主要体现在两方面。其一，系统不再局限于固定语音播报，而是支持定制化输出。其二，音色样本与克隆状态被纳入系统配置管理，使多音色场景具备可维护性。

结合项目测试代码可以看出，音色模块还对克隆失败、旧 URI 清理、文件格式异常和服务端 404 回退等情况做了处理。这说明系统在音色能力设计时并非只关注“成功路径”，而是兼顾了异常处理和工程健壮性。

音色管理页面同时覆盖系统音色、样本上传、克隆与试听，其界面如图 5-8 所示。



图 5-8 音色管理页面

后台在接收样本后先做格式与时长校验，再决定是否调用音色克隆接口：

```
clone_sample_error = _validate_clone_sample(target, suffix)
config["sample_source"] = str(target)
if clone and not clone_sample_error:
    clone_uri = await siliconflow_upload_reference_voice(
        db=db, model=settings.siliconflow_tts_model,
        custom_name=_safe_custom_name(row.name), reference_text=used_reference_text,
        audio_bytes=audio_bytes, filename=target.name
    )
```

5.10 实时 WebSocket 服务设计

考虑到实时语音链路中“说话、响应、插话打断、重新接续”较难仅凭文字传达，本文进一步给出实时对话与打断交互示意图，如图 5-10 所示。该图用于解释交互状态变化，不替代真实测试截图。



图 5-10 实时语音对话与打断交互示意图（AI 示意图）

后端实时 WebSocket 服务是语音实时链路落地的关键。通过 /ws/realtime-chat 接口，系统能够持续接收音频块、判断轮次边界、触发局部识别、提交整句文本，并在回复生成后把文本和音频事件实时回传给前端。

在实现细节上，服务端会维护当前轮次任务、局部识别任务、最近局部文本、音频格式以及去重窗口等状态，并对重复提交、空提交和并发冲突做额外判断。局部识别结果以 user.transcript.partial 事件返回前端，最终生成结果则进一步写入消息表和记忆表。这样既保证了实时交互体验，也减少了高频音频输入下的状态错乱问题。

系统配置页面集中展示数据库状态、异常策略和高危清理操作，其界面如图 5-9 所示。

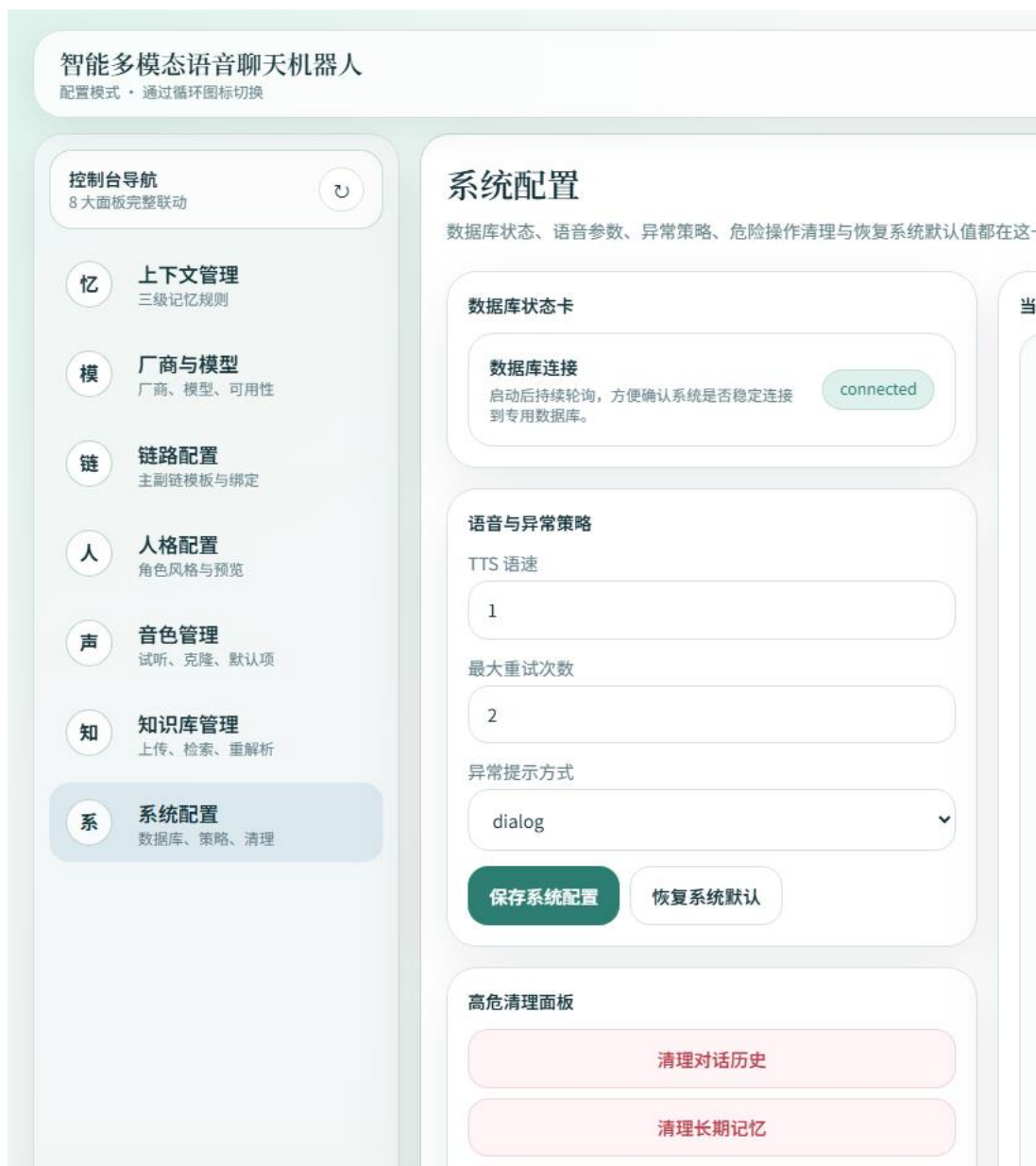


图 5-9 系统配置页面

实时 WebSocket 服务通过局部识别任务、轮次任务和持久化逻辑配合完成实时通话闭环：

```
await websocket.send_json({"event": "session.ready", "session_id": str(session_id),
"mode": "integrated-realtime"})
prepared_turn = await prepare_assistant_turn(db, session, text_input)
prepared = await prepare_realtime_chain(db, session_id=session_id,
text_input=text_input, context_text=prepared_turn.context_hint,
knowledge_text=prepared_turn.knowledge_text)
await recompute_session_memory(db, session_id)
```

5.11 测试用例对应的设计验证

结合项目中的自动化测试代码可以看出，前述设计并不只是停留在论文描述层面，而是已经以较完整的工程形式落地。测试覆盖了基础服务、默认配置、消息与记忆流、知识上传与检索、实时空提交以及音色克隆等场景，这些内容能够从侧面说明系统在关键路径上具备可验证性。

健康检查接口测试验证了系统基础服务可用性。

默认通话配置测试验证了系统在启动后能够自动准备实时链路默认模型配置。

文本消息与记忆流测试验证了会话、消息与上下文记忆模块之间的联动。

知识上传与检索测试验证了知识库文档上传和检索增强链路的可用性。

实时空提交测试验证了语音实时链路在“无有效语音输入”场景下的状态处理逻辑。

音色克隆相关测试验证了回退逻辑、错误提示和样本校验机制。

对于本科毕业设计而言，这类自动化测试不仅用于发现问题，也为答辩阶段说明系统完成度、模块联动关系和异常处理能力提供了较直接的支撑。

5.12 关键设计特点总结

综合本章内容可以看出，系统详细设计并不是把多个功能简单叠加，而是围绕统一会话运行时组织前端交互、链路策略、知识增强和配置管理。

一方面，统一会话运行时保证了不同交互方式在状态管理和事件接口上的一致性；另一方面，双语音链路、知识增强、记忆管理以及音色配置又保留了足够的可替换性和扩展空间。这种“高层抽象稳定、底层策略可调整”的设计思路，是本文系统能够同时兼顾实时体验与工程可控性的关键。

6 系统测试

6.1 测试目标

系统测试的目的，主要是验证智能语音聊天机器人在功能完整性、交互流程、链路切换、知识增强和个性化配置等方面是否达到设计要求。考虑到本课题的重点是系统设计与实现，测试工作以功能验证和流程验证为主，同时补充必要的对比分析，用来说明系统在不同链路和不同配置下的表现差异。

6.2 测试环境

本次测试主要在项目开发环境下完成，重点关注前后端联动、数据库读写、浏览器端音频采集以及多 Provider 配置下的接口调用情况。测试环境如表 6-1 所示。

表 6-1 系统测试环境

项目	配置说明
前端环境	React + Vite + TypeScript 开发模式
后端服务	FastAPI + Uvicorn
数据库	PostgreSQL + pgvector
浏览器环境	支持麦克风采集与 WebSocket 通信的现代浏览器
模型服务	按系统 Provider 配置接入文本、ASR、TTS 与实时模型接口

由表 6-1 可知，本次测试环境以项目实际开发环境为基础，没有额外搭建复杂的实验平台。这种测试口径更接近系统当前的真实运行条件，因此得到的结果更适合用于说明系统是否具备稳定的工程闭环。

6.3 测试内容

6.3.1 文本对话功能测试

文本对话测试主要检查会话创建、消息保存、助手回复生成和历史记录展示是否正常，验证系统最基本的问答闭环是否成立。

6.3.2 语音输入功能测试

语音输入测试主要观察浏览器识别和后端识别两条路径是否都能把语音稳定转为文本，并继续完成后续问答流程。

6.3.3 实时通话功能测试

实时通话测试重点检查音频块持续上传、局部转写回传、助手播报以及插话打断等关键环节是否能够连续工作。

6.3.4 知识增强功能测试

知识增强测试主要验证文档上传、切片存储、检索召回和回答增强是否形成闭环，尤其关注系统在项目资料问答场景下能否给出更贴近文档内容的回答。

6.3.5 音色管理功能测试

音色管理测试则围绕音色创建、样本上传、格式校验、克隆与预览展开，重点检查成功路径和异常路径下的反馈是否清晰。

6.4 测试用例设计

为便于整理测试结果，本文将测试内容划分为功能测试和对比测试两部分。其中，功能测试用于确认系统各主要模块是否可正常运行；对比测试用于观察不同链路和不同知识配置下的表现差异。

6.4.1 功能测试用例

安徽工业大学毕业设计（论文）说明书

功能测试结果如表 6-2 所示。整体来看，核心模块均能够完成设计目标，没有出现阻断系统使用的关键故障。

表 6-2 功能测试结果汇总

测试项	输入条件	实际情况	结论
创建新会话	用户新建会话	系统成功返回会话标识并更新会话列表	通过
文本问答	输入普通文本问题	能够保存消息并生成助手回复	通过
知识文档上传与检索	上传文档后发起检索测试	文档完成切片存储，能够召回相关片段	通过
音色样本上传与克隆	上传符合要求的样本音频	成功路径可触发克隆，异常格式会返回错误提示	通过

从表 6-2 可以看出，文本问答、知识文档处理和音色管理等核心功能均能按预期完成，说明系统已经具备较完整的基础功能集。对于本科毕业设计而言，这意味着系统不只是实现了局部功能，而是形成了可以实际运行和演示的整体。

6.4.2 对比与流程测试

对比与流程测试结果如表 6-3 和表 6-4 所示。该部分既关注跨模块协同是否顺畅，也关注不同配置下的实际体验差异。

表 6-3 不同语音链路的测试观察

对比项	浏览器 ASR 分阶段链路	后端 ASR 分阶段链路	一体化实时语音链路
启动方式	浏览器直接识别，启动较快	先录音再上传，链路更长	建立 WebSocket 持续传输
实时性	适合短语音输入场景	响应略慢，但流程清晰	实时性最好，插话体验更自然
兼容性	受浏览器能力影响较大	兼容性较好，依赖后端服务稳定性	对网络与服务协同要求较高
适用场景	轻量语音输入	可控性优先的测试或实验场景	强调连续对话体验的实时通话场景

由表 6-3 可以看出，不同语音链路各有侧重。浏览器 ASR 分阶段链路更轻量，后端 ASR 分阶段链路更便于控制，一体化实时语音链路则在连续交互和打断体验上更有优势。这一结果也验证了本文保留双链路设计的必要性。

表 6-4 知识库开启与关闭的问答对比

测试场景	关闭知识库	开启知识库	观察结论
通用聊天问题	能够正常回答，但内容偏通用	回答差异不明显	通用问题对知识库依赖较低
项目配置相关问题	回答容易停留在泛化表述	能结合项目资料给出更具体说明	知识库可提高细节贴合度
文档细节查询	细节覆盖不足，容易遗漏关键信息	更容易命中文档片段并给出针对性回答	知识增强对资料型问答更有效

表 6-4 反映出，知识库是否开启会直接影响资料型问答的表现。面对通用聊天问题时，两种状态差异不大；但当问题涉及项目配置和文档细节时，开启知识库后的回答通常更具体，也更容易贴近原始资料。

6.5 测试结果分析

从功能测试结果看，系统的核心流程能够按照预期运行。文本对话、会话创建、消息保存、知识库上传与检索等基础能力均能形成闭环，说明后端接口、数据库持久化和前端状态更新之间的协同关系基本稳定。

从语音交互测试结果看，分阶段语音链路和一体化实时语音链路各有适用场景。分阶段链路便于观察 ASR、RAG、LLM 和 TTS 各阶段结果，适合作为调试与分析链路；一体化实时链路更接近自然通话体验，能够通过持续连接降低交互等待感，但对网络、音频采集和服务端协同提出了更高要求。

从知识增强测试结果看，开启知识库后，系统在回答与项目配置、文档细节和专有资料相关的问题时更加具体。关闭知识库时，模型仍能回答通用问题，但面对私有资料时更容易给出泛化表述。该结果说明 RAG 模块能够提升资料型问答的针对性。

从异常场景测试结果看，系统对空语音输入、音色克隆失败、服务端 404 回退、知识库无命中等情况进行了处理。虽然这些测试不能覆盖所有真实用户场景，但能够说明系统在关键路径上具备基本容错能力。

从测试结果看，系统已经形成较完整的基础闭环。文本问答能够稳定完成会话创建、消息存储与回复生成；语音输入链路在浏览器能力不足时可以回退到后端识别路径；实时通话链路能够通过 WebSocket 和事件驱动机制完成局部转写、回复播报和插话打断；知识增强与音色管理模块也能够各自场景下提供有效支撑。

不过，测试结果也说明系统仍然受到外部条件影响。语音链路对浏览器兼容性、网络状况和模型服务稳定性较为敏感；复杂噪声环境下的识别效果仍可能波动；不同 Provider 的响应时间差异也会直接影响实时通话体验。因此，本章测试更适合说明系统已经具备可运行、可验证和可分析的工程基础，而不是把它视为一个已经完全成熟的产品。

6.6 测试不足与改进方向

在现有测试基础上，系统已经完成了主要功能验证，但若想让测试结论更完整，后续仍有几个方向值得继续补充。

（1）增加不同语音链路在首字延迟、首音延迟和整体响应时间上的定量对比，从而更直观地说明分阶段链路与时延链路各自的适用场景。

（2）围绕知识库开启与关闭两种状态设计更细致的对比实验，分析检索增强对回答准确性、细节覆盖和可解释性的影响。

（3）补充浏览器 ASR 与后端 ASR 在典型语句、噪声环境和长句场景中的识别对比，为后续链路选型提供更直接的数据依据。

（4）补充音色克隆成功与失败场景的运行截图和测试记录，使个性化能力部分的结论更具说服力。

这些补充方向并不影响当前系统已经完成的基本功能闭环，但它们能够让论文中的测试部分从“功能可用”进一步走向“结果更充分、分析更扎实”。

6.7 本章小结

本章测试表明，本文实现的智能语音聊天机器人已经具备较完整的多模态交互能力。系统能够完成文本问答、语音输入、实时通话、知识增强和音色管理等核心流程，整体上达到了本科毕业设计对“可运行、可分析、可扩展”的基本要求。

结 论

本文围绕“智能语音聊天机器人设计与实现”这一课题，完成了一套面向中文场景的多模态智能对话系统。系统以统一会话运行时为核心，将文本输入、语音输入和实时语音通话组织到同一套会话框架之中，并在此基础上接入知识增强、上下文记忆、人格配置和音色管理等能力，形成了较完整的系统结构。

从系统设计和实现结果来看，本文较好地完成了预定目标。一方面，统一会话运行时保证了不同交互方式在状态管理和事件处理上的一致性；另一方面，分阶段语音链路与一体化实时语音链路并行存在，使系统能够根据场景在可控性和实时性之间进行选择。与此同时，后端通过知识检索、模型路由、WebSocket 编排和数据库持久化等机制，支撑了系统在工程上的完整落地。

需要说明的是，本文系统仍存在一定不足。例如复杂噪声环境下的语音识别稳定性还有提升空间，长时间连续对话中的记忆召回策略仍可进一步优化，不同 Provider 之间的时延差异也会影响实时通话体验。这些问题说明本系统已经具备较好的实验和应用基础，但距离更加稳定、更加精细的产品级表现仍有差距。

后续工作可以从三个方向继续推进：其一，补充更细粒度的性能指标采集与链路对比实验，使系统评估更加充分；其二，继续优化实时交互过程中的异常恢复和自动回退策略，提升复杂环境下的稳定性；其三，结合更具体的应用场景扩展多模态输入输出能力，例如图像理解、视频分析或更丰富的情感化语音生成。

总体来看，本文完成的智能语音聊天机器人系统已经实现了课题设定的主要任务，也为后续继续完善语音交互、知识增强和个性化对话能力打下了较好的工程基础。

参考文献

- [1] Radford A, Kim J W, Xu T, et al. Robust speech recognition via large-scale weak supervision[EB/OL]. (2022-12-06)[2026-04-27]. <https://arxiv.org/abs/2212.04356>.
- [2] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems. 2017: 5998-6008.
- [3] Lewis P, Perez E, Piktus A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks[C]//Advances in Neural Information Processing Systems. 2020, 33: 9459-9474.
- [4] Borgeaud S, Mensch A, Hoffmann J, et al. Improving language models by retrieving from trillions of tokens[C]//Proceedings of the 39th International Conference on Machine Learning. PMLR, 2022, 162: 2206-2240.
- [5] Zhao W X, Zhou K, Li J, et al. A survey of large language models[EB/OL]. (2023-03-31)[2026-04-27]. <https://arxiv.org/abs/2303.18223>.
- [6] Gao Y, Xiong Y, Gao X, et al. Retrieval-augmented generation for large language models: A survey[EB/OL]. (2023-12-18)[2026-04-27]. <https://arxiv.org/abs/2312.10997>.
- [7] Gao Z, Li Z, Wang J, et al. FunASR: A fundamental end-to-end speech recognition toolkit[C]//Proc. Interspeech 2023. 2023: 1593-1597.
- [8] Zhong W, Guo Y, Gao N, et al. MemoryBank: Enhancing large language models with long-term memory[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2024, 38(17): 19724-19731.
- [9] Kim J, Kong J, Son J. Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech[C]//Proceedings of the 38th International Conference on Machine Learning. PMLR, 2021, 139: 5530-5540.
- [10] Fette I, Melnikov A. The WebSocket Protocol: RFC 6455[S]. IETF, 2011.
- [11] The PostgreSQL Global Development Group. PostgreSQL Documentation[EB/OL]. [2026-04-27]. <https://www.postgresql.org/docs/current/>.
- [12] pgvector. pgvector: Open-source vector similarity search for Postgres[EB/OL]. [2026-04-27]. <https://github.com/pgvector/pgvector>.

致 谢

在本论文的选题、系统设计、实现与写作过程中，我得到了指导教师王朋飞老师的悉心指导。老师在课题方向确定、系统方案梳理、论文结构组织和内容修改等方面都给予了耐心帮助，使我能够顺利完成本次毕业设计（论文）。

同时，感谢学院老师在本科阶段的培养与帮助，感谢同学们在项目交流和测试反馈中的支持，感谢家人对我学习和毕业设计工作的理解与鼓励。谨以此文向所有给予我帮助和支持的人表示衷心感谢。

附录 1 部分主要源代码

本附录列出系统关键实现文件，定稿时可根据指导教师要求继续补充完整代码清单。为保持论文正文简洁，正文部分仅描述关键设计思想，不展开大段源代码。

前端统一会话运行时核心文件：`frontend/src/runtime/voice/sessionRuntime.ts`。该文件负责维护会话标识、运行状态、转写文本、消息列表和性能指标，并向页面层发布统一事件。

前端分阶段语音链路核心文件：`frontend/src/runtime/voice/splitStrategy.ts`。该文件负责组织浏览器 ASR、后端 ASR、文本提交、语音播报和插话打断等流程。

前端一体化实时语音链路核心文件：`frontend/src/runtime/voice/integratedStrategy.ts`。该文件负责 `WebSocket` 连接、音频分片发送、实时事件接收、PCM 播放和用户插话检测。

后端聊天服务核心文件：`backend/app/modules/chat/service.py`。该文件负责会话创建、消息保存、上下文组织、知识增强调用和模型回复生成。

后端实时通信核心文件：`backend/app/modules/realtime_ws/router.py`。该文件负责实时语音 `WebSocket` 接入、音频片段处理、局部转写事件、回复生成和链路审计。

后端数据模型核心文件：`backend/app/models/entities.py`。该文件定义会话、消息、记忆、`Provider`、模型目录、人格、音色、链路配置、知识库和审计等数据表结构。